UNIT-1: JSON

- JSON stands for **J**ava**S**cript **O**bject **N**otation.
- JSON is a lightweight format for storing and transporting data.
- JSON is often used when data is sent from a server to a web page.
- JSON is "self-describing" and easy to understand.
- A JSON object is an unordered set of name/value pairs.

Why Use JSON?

The JSON format is syntactically similar to the code for creating JavaScript objects. Because of this, a JavaScript program can easily convert JSON data into JavaScript objects.

Since the format is text only, JSON data can easily be sent between computers, and used by any programming language.

JavaScript has a built in functions

1. Converting JSON strings into JavaScript objects: **JSON.parse()**
2. converting an object into a JSON string: **JSON.stringify()**

JSON Files

- The file type for JSON files is ".json"
- The MIME type for JSON text is "application/json"

Characteristics of JSON

- JSON is easy to read and write.
- It is a lightweight text-based interchange format.
- JSON is language independent.

JSON supports the following two data structures:

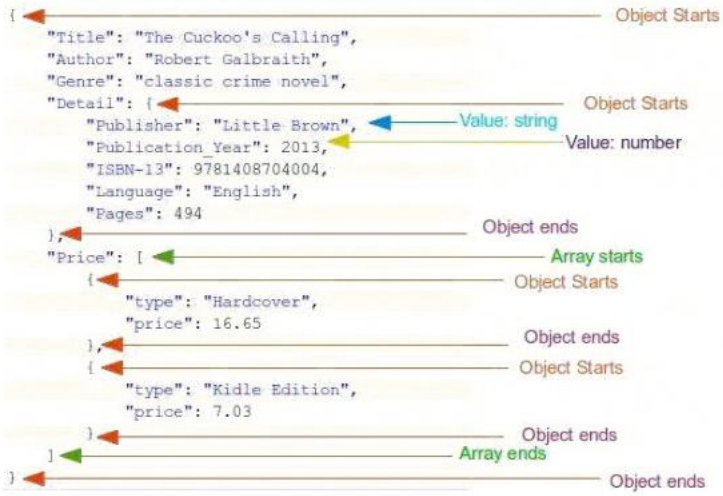
 Collection of name/value pairs: This Data Structure is supported by different programming languages.

 Ordered list of values: It includes array, list, vector or sequence etc.

JSON Syntax Rules

JSON syntax is derived from JavaScript object notation syntax:

- Data is in name(key)/value pairs.
- Data is separated by commas "**,**".
- Curly braces hold objects and each name is followed by '**:**'(colon), the name/value pairs are separated by , (comma). **{"Key": "Value"}**
- Square brackets hold arrays and values are separated by ,(comma).



<u>Example:1</u>

var Obj= {"marks":97}

console.log(Obj) ----------> {marks: 97}
console.log(Obj.marks) ----------> 97

<u>Example:2</u>

var Obj= {"name":"PKP","marks":97,"dist":true}

console.log(Obj) ----------> {name: 'PKP', marks: 97, dist: true}
console.log(Obj.name) ----------> PKP
console.log(Obj.marks) ----------> 97
console.log(Obj.dist) ----------> true

Example of a JSON string: '{"name":"ABC", "age":20}' - It defines an JSON string with 2 properties: Name and age

Example of a JSON object: { "name":"ABC", "age":20 } OR { name:"ABC", age:20 } - It defines an JSON object with 2 properties: Name and age

JSON - DATATYPES

In JSON, values must be one of the following data types:

- a string
- a number
- an object (JSON object)
- an array
- a boolean
- *null*

JSON values **cannot** be one of the following data types:

- a function
- a date
- *undefined*

JSON Strings
Strings in JSON must be written in double quotes.
Example
{"name":"LJU"}

JSON Numbers
Numbers in JSON must be an integer or a floating point.
Example
{"age":30}

JSON Objects
Values in JSON can be objects.
Example
var obj= {
"employee":{"name":"LJU", "age":30, "city":"Ahmedabad"}
}

JSON Arrays
Values in JSON can be arrays.
Example
{
"employees":["John", "Anna", "Peter"]
}

JSON Booleans
Values in JSON can be true/false.
Example
{"sale":true}

JSON null
Values in JSON can be null.
Example
{"middlename":null}

Since the format is text only, JSON data can easily be sent between computers, and used by any programming language.

Methods to Access Data in JSON / Objects

Method	Explanation	Syntax
Dot Notation	Access a value directly using the key with a dot (`.`). It is the most common and simplest way when the key name is known.	`object.key`
Bracket Notation	Access a value using square brackets. This method is useful when the key name is stored in a variable or when the key contains spaces or special characters.	`object["key"]`
Nested Object Access	Used when one object exists inside another object. Multiple dots are used to reach the inner value.	`object.key.key`
Array Inside Object	Used when an array is stored inside an object. Access the array using the key and then use the index number to get a specific element.	`object.array[index]`

Execution Methods

Using HTML FILE

J1.html

```
<script>
var a= {
  "name": "Test",
   “age”:20
}
Console.log(a.name)
</script>
```

Example

Task1.html

```
<script>

let userProfile = {"userId": 101, "fullName": "Khushbu Patel", "email": "pkp@email.com",
        "isPremiumMember": true,
        "hobbies": ["Reading", "Cooking", "Coding"],
        "contact": {"phone": "+91-9876543210","city": "Ahmedabad","state": "Gujarat"}
        }

console.log(userProfile.fullName);  // dot notation
console.log(userProfile["email"]); // bracket notation
console.log(userProfile.contact.city);  // Nested Object Access
console.log(userProfile.hobbies[1]);  // Array Inside Object
</script>
```

Output

Khushbu Patel
pkp@email.com
Ahmedabad
Cooking

Using JSON FILE

Create a json file named j1.json (Strict formation require)

```
{
  "name": "Test",
   “age”:20
}
```

1.js file

```
const data = require("./j1.json");  // Import JSON file
console.log(data.name);
```

To run enter command node 1.js in terminal

JSON
USING JS FILE

j1.js

```
var a= {
  "name": "Test",        // String
  "age": 25,             // Number

  "ispass": false,     // Boolean
  "address": {           // Object (Nested JSON)
    "city": "Ahmedabad",
    "zip": "380015"
  },
  "subjects": ["Math", "Science"],  // Array
  "year": null   // Null
}

console.log(a);
console.log(a.age)
console.log(a['address'])
console.log(a['address']['city'])
```

Output:

```
{
  name: 'Test',
  age: 25,
  ispass: false,
  address: { city: 'Ahmedabad', zip: '380015' },
  subjects: [ 'Math', 'Science' ],
  year: null
}
25
{ city: 'Ahmedabad', zip: '380015' }
Ahmedabad
```

To run enter command node j1.js in terminal

Parse & Stringify

JSON.parse()

JavaScript has a built in function for converting JSON strings into JavaScript objects:

When receiving data from a web server, the data is always a string. Widley used to Convert API response (string) → JavaScript object

Parse the data with JSON.parse(), and the data becomes a JavaScript object.

We received this text from a web server: '{"var1":"LJ","var2":"University"}'

Use the JavaScript function **JSON.parse(string) to convert it into an object**.

Example:

```
<script>
let obj = JSON.parse('{"var1":"LJ","var2":"University"}');
console.log(obj);
console.log(obj.var1 + " " + obj.var2);
</script>
```

Output:

{var1: 'LJ', var2: 'University'}

LJ University

JSON.stringify()

JavaScript has a built in function for converting objects into JavaScript strings:

When sending data to a web server, the data has to be a string. Widley used to Convert JavaScript object → JSON string for API request

Convert a JavaScript object into a string with JSON.stringify(object).

Example:

```
<script>
let obj = JSON.stringify({var1: 'LJ', var2: 'University'});
console.log(obj);
</script>
```

Output:

'{"var1":"LJ","var2":"University"}'

Note:

- One of the most common uses for JSON is when using an API, both in requests and responses.
- JSON.parse(string) takes a string of valid JSON and returns a JavaScript object.

- The inverse of this function is JSON.stringify(object) which takes a JavaScript object and returns a string of JSON, which can then be transmitted in an API request or response.

Example of an array of objects

```
var cars = [{
   color: 'gray',
   model: '1',
   nOfDoors: 4
   },
   {
   color: 'yellow',
   model: '2',
   nOfDoors: 4
}];
```

var jsonCars = JSON.stringify(cars);

```
"[{"color":"gray","model":"1","nOfDoors":4},{"color":"yellow","model":"2","nOfDoors":4}]"
```

Program:

```
<html>
<script>
const user='{"name":["ABC","PKP","XYZ"],"age":"28"}'; //value is in String formation
console.log(user)  //Print string formation -----> {"name":["ABC","PKP","XYZ"],"age":"28"}
console.log(user.name) // Undefined

obj=JSON.parse(user) //convert string data into objects
console.log(obj) // obje is object using parse --->{name: Array(3), age: '28'}
console.log(obj.name) // ---> Detail of name --->(3) ['ABC', 'PKP', 'XYZ']
obj4=JSON.stringify(obj) // convert into string again
console.log(obj4) // ----------->{"name":["ABC","PKP","XYZ"],"age":"28"}
document.write(obj.name) // To wite in Chrome
</script>
</html>
```

Updating Values of Existing Object Keys

- **Dot Notation:**
  Dot notation is used to update the value of an existing key in an object using the dot (.) operator. It is the simplest and most commonly used method when the key name is known and does not contain spaces or special characters.

  Syntax: object.key = newValue

- **Bracket Notation:**
  Bracket notation is used to update the value of a key using square brackets. This method is useful when the key name contains spaces, special characters, or when the key needs to be accessed dynamically. The key name is written inside quotes within the brackets.

  Syntax: object["key"] = newValue

- **Using Variable Key:**
  This method is used when the key name is stored in a variable. Instead of writing the key directly, the variable containing the key name is placed inside square brackets to update the corresponding value in the object.

  Syntax: object[variable] = newValue

Example:

```
let person = {name:"abc",age: 17, city:"Ahmedabad"};

person.age = 30;                // update value of age using Dot Notation
console.log(person);            //change in person value

person["name"] = "xyz"          //update value of name using Bracket Notation
console.log(person);            //change in age value

let key = "city";               // Update value Using Variable Key
person[key] = "Gandhinagar";
console.log(person);            // change in city value
```

Output:

```
{ name: 'abc', age: 30, city: 'Ahmedabad' }
{ name: 'xyz', age: 30, city: 'Ahmedabad' }
{ name: 'xyz', age: 30, city: 'Gandhinagar' }
```

Method to create JSON object

Syntax:

obj[key]=value

eg.

```
var obj2={}
obj2[“a”]=111
console.log(obj2)
```

output:

{“a”:111}

Examples

- **Write code to fetch the different values and also “Give again” value from user object.**

```
const user={
   "name":"abc",
   "age":"28",
   "course":["FSD-1","DE","FSD-2"],
   "adress":["T1","t2",{"t3":"Give again"}]
}
console.log(user.name[0])  // ---> a
console.log(user.age)  // ---> 28
console.log(user.course) // ---> (3) ['FSD-1', 'DE', 'FSD-2']
console.log(user.adress[1]) // ---> t2
console.log(user.adress[2]) // --->{t3: 'Give again'}
console.log(user.adress[2].t3) // ---> Give again
```

- **Example on Nested JSON (.html)**

```
<html>
<script>
const sub={
        "FSD":[
                {   "Topic":"HTML",
                "course":"Beginer",
                 "content":{"tags":"table","form":"xyz"}
                },
                { "Topic":"CSS",
                 "course":"Beginer",
                 "content":["tags","table","form"]}
                 ]
                 };
console.log(sub.FSD[0].content.tags); // ---> table
console.log(sub.FSD[1].content[0]); // ---> tags
</script></html>
```

- **From below JSON object fetch the values as asked.**

```
<html>
<body>
   <script >
    const a= { "Datastructures":  [
                                {"Name": "tree", "course":"Intro","content":["1","B","C"]},
                                { "Name": "tree1","course":"Intro1", "content":["1","B","C","d"]}
                                            ] ,
                        "xyz": { "Name":"Graphics", "Topic":["BFS","CDF","Sorting"],
                }

     console.log(a.Datastructures[1].Name); //tree1
     console.log(a.Datastructures[0].Name); //tree
     console.log(a.xyz.Name);//Graphics
     console.log(a.xyz.Topic); //['BFS', 'CDF', 'Sorting']
     console.log(a.xyz.Topic[0]); //BFS
     console.log(a.Datastructures[1]); //   {  "Name": "tree1",  "course":"Intro1",
"content":["1","B","C","d"]  }
     console.log(a.Name); //undefined
     console.log(a.xyz); //{Name: 'Graphics', Topic: Array(3)}
     </script>
</body>
</html>
```

Consider below JSON object and fetch values using for loop (for…in/for…of).

```
const sub = {
"FSD": [ { "Topic": "HTML", "course": "Beginer", "content": ["tags", "table", "form"], },
            { "Topic": "CSS", "course": "Beginer", "content": ["tags", "table", "form"] }
          ] };
```

Solution

```
<script>
const sub =
{ "FSD": [ {
"Topic": "HTML",
"course": "Beginer", "content": ["tags", "table", "form"], },
{
"Topic": "CSS",
"course": "Beginer",
"content": ["tags", "table", "form"]
} ] };
document.write("// Using for…in loop <br>")
```

```
// Using for…in loop
for (x in sub.FSD) {
for (y in sub.FSD[x]) {
document.write(sub.FSD[x][y] + "<br>")
} }
document.write("// Using for…of loop <br>")
// Using for…of loop
for(x1 of sub.FSD) {
document.write(x1.Topic + "<br>")
document.write(x1.course + "<br>")
document.write(x1.content + "<br>")
console.log(x)
}
</script>
```

Output:

```
// Using for…in loop
HTML
Beginer
tags,table,form
CSS
Beginer
tags,table,form
// Using for…of loop
HTML
Beginer
tags,table,form
CSS
Beginer
tags,table,form
```

New date object Task

Write one JSON string with date property (yyyy-mm-dd) and print date in India standard time.

```
<script>
const obj=JSON.parse(`{"name":"xyz","age":"17","dob":"1997-03-14"}`);
console.log(obj.dob);
a = new Date(obj.dob);
console.log(a);
</script>
```

Output

```
Fri Mar 14 1997 05:30:00 GMT+0530 (India Standard Time)
```

Write one JSON string with date property (yyyy-mm-dd) and print date in India standard time While clicking on button..

```
<html>
<script>
function call()
{
   text='{"Name":"ABC","Age":18,"City":"Ahmedabad","DOB":"1990-08-24"}';
   obj=JSON.parse(text);
   console.log(obj.DOB);
   ob=new Date(obj.DOB)
   document.getElementById("demo").innerHTML= "Birth day of "+ obj.Name +" is "+ ob;
}
</script>
<body>
   <P id="demo">  </P>
   <button onclick="call()"> Click Me </button>
</body>
</html>
```

Output:

Birth day of ABC is Fri Aug 24 1990 05:30:00 GMT+0530 (India Standard Time)

Click Me

Note: Date Objects are directly not allowed in JSON. If you need to include date, write it as string.

- **Write a function 'fromListToObject' which takes in an array of arrays, and returns an object with each pair of elements in the array as a key-value pair.**

Hint:

Example input:
[['make', 'Ford'], ['model', 'Mustang'], ['year', 1964]]

Function's return value (output):
{make : 'Ford'model : 'Mustang',year : 1964}

Do not change the input string. Assume that all elements in the array will be of type 'string'.

Note that the input may have a different number of elements than the given sample.
For instance, if the input had 6 values instead of 4, your code should flexibly accommodate that.

Solution:

```
var array = [['make', 'Ford'], ['model', 'Mustang'], ['year', 1964]]
function fromListToObject(array) {
        emptyObject = {}
        for(var i = 0; i < array.length; i++){
        var newArray = array[i];
        emptyObject[newArray[0]] = newArray[1];
        }
                return emptyObject;
}
var object = fromListToObject(array);
console.log(object);
```

Output

```
{make: 'Ford', model: 'Mustang', year: 1964}
```

- **Write a function 'transformFirstAndLast' that takes in an array, and returns an object with:**
  1) the first element of the array as the object's key, and
  2) the last element of the array as that key's value.

Hint:

Example input:

['Queen', 'Elizabeth', 'Of Hearts', 'Beyonce']

Function's return value (output):

{Queen : 'Beyonce'}

Solution:

```
function transformFirstAndLast(array) {
var object = {};
object[array[0]] = array[array.length-1];
return object;}
var arrayList = ['Queen', 'Elizabeth', 'Of Hearts', 'Beyonce'];
console.log(transformFirstAndLast(arrayList));
```

Output :

{Queen: 'Beyonce'}

Explaination:

array**[0]** → Accesses the **first element** of the array and treats it as a **key**.

array**[array.length - 1]** → Accesses the **last element** of the array and assigns it as a **value**.

object**[array[0]] = array[array.length - 1];** → Dynamically assigns the key-value pair in the object.

- **Write a JS to store an array of objects having name and age. display name and age of person with highest age.**

```
const person =[{name: "PQR", age: 38},{name: "ABC", age: 35},{name: "XYZ", age: 47}]
```

Solution:

```
const person = [{name: "PQR", age: 38}, {name: "ABC", age: 35}, {name: "XYZ", age: 47}]
function getHighest(person) {
   return person.sort((a, b) => b.age - a.age)[0];
}
console.log(getHighest(person));
```

Output:
{ name: 'XYZ', age: 47 }

- **Create a JSON object with two divisions, where each division has a 'name' field containing an array of employee initials. These names should be sorted alphabetically in the object and should be written to the file. At last, both division objects should be visible with names sorted alphabetically in file.**

```
var test = { "division1": {"name":["Z","B","H"]},"division2": {"name" :["Y","A","G"]}}
const div1_data = test.division1.name;
const div2_data = test.division2.name;
var combine_data = div1_data.concat(div2_data).sort();
console.log(combine_data);
```

Output: ['A', 'B', 'G', 'H', 'Y', 'Z']

- **Write a JSON object which contains array of 3 objects. Each object contains 2 properties name and height. Now, sort an array values by height in descending order. Print name in terminal as per the sorted height.**

```
const student = [{name: "ABC", height: 5.7},{name: "PQR", height: 6.0},
 {name: "XYZ", height: 6.2} ]
const ss = student.sort((a, b) => b.height - a.height);
// Print name and height of each student
for (const person of ss) {
  console.log(`${person.name}: ${person.height}`);
}
```

Output

XYZ 6.2
PQR 6
ABC 5.7

- **Get JSON object values using for loop and make HTML table to display these values.**

```
const sub ={"FSD": [  {        "Topic": "HTML",
                               "course": "Beginer",
                               "content": ["tags", "table", "form"],
                         },
                       {       "Topic": "CSS",
                               "course": "Beginer",
                               "content": ["tags", "table", "form"]
                       }
                       ]};
var temp = "<table border='2px'>";
temp += "<tr>";
for (x in sub.FSD[0]) {
temp += "<th>" + x + "</th>";
}
temp += "</tr>";
for (x in sub.FSD) {
console.log(x); temp += "<tr>";
for (y in sub.FSD[x]) {
temp += "<td>" + sub.FSD[x][y] + "</td>";
}
temp += "</tr>";
}
temp += "</table>";
document.write(temp);
```

Topic	course	content
HTML	Beginer	tags,table,form
CSS	Beginer	tags,table,form

Write a code to fetch the values of keys and print a sentence as "Hi! We are students of semester 4 of CSE branch" in h1 tag on browser.

```
<script>
const a = {
 "A" : "LJU",
"B" : ["CSE", "IT" , "CE"],
"C" : [ {
          "D" : "Hi",
          "E" : ['are', 4, {'F' : ['semester', 'We']} ]
          } ],
"G" : { "H" : "students", "I" : ["of","!"] },
"J" : [{"K" :"Python", "L":"branch"},"FSD-2"]
}
document.write(a.C[0].D, a.G.I[1] + " " + a.C[0].E[2].F[1] +" " + a.C[0].E[0] + " " +a.G.H + "
" +a.G.I[0] + " "+ a.C[0].E[2].F[0] + " "+ a.C[0].E[1] + " " +a.G.I[0] + " "+ a.B[0] + " "+
a.J[0].L)
</script>
```

Task

- **Print following statement in console from given JSON objects.**

```
 const a = {"Name" : "Ramesh",
"Subects" : ["Maths", "Scence" , "chemistry"],
"Grade" : {"Type" : "marks","Total" : [88,90,99,87]},
"Range" : {"opt" : "100", "type" : ["secure","subject","class"]},
"achive" : [{"Rank" :"rank", "place":[1,2,3]}, {"Ordinalindicator":"st"},"12"],
"joints" : ['outof','got','and']
}
```

Output: Ramesh got 99 outof 100 marks
and secure 1st rank.

- **Make a statement : "John has BMW- X3 at the age of 30" in web browser**

```
myObj = {
          "name" : "John",
          "age" : 30,
          "cars" : [
                    { "name" : "Ford",  "models":[ "Fiesta", "Focus", "Mustang" ] },
                    { "name" : "BMW", "models" : [ "320", "X3", "X5" ] },
                    { "name" : "Fiat", "models" : [ "500", "Panda" ] }
          ] }.
```